

Проверка статистических гипотез

6.1. Цель работы:

Ознакомиться с некоторыми статистическими тестами, принципами их работы. Научиться оценивать нормальность распределения выборки, а также выполнять оценку статистических гипотез.

6.2. Общие сведения

Любой вид анализа данных требует (после этапа сбора этих данных) выдвижения некоего предположения (гипотезы) относительно характеристик выбранной совокупности.

Статистической гипотезой называют любое предположение о виде или свойствах распределения генеральной совокупности. Мы будем рассматривать две гипотезы: **нулевую (основную) H_0** и **альтернативную (конкурирующую) H_1** .

Нулевая гипотеза H_0 подлежит проверке, по результатам которой ее можно принять либо отклонить. «Принять» означает «не получить убедительных аргументов для отклонения гипотезы».

Альтернативная гипотеза H_1 принимается только тогда, когда есть убедительное статистическое доказательство для отклонения основной гипотезы.

Ошибкой первого рода называют ситуацию, в которой отвергается верная гипотеза H_0 . При **ошибке второго рода** принимается гипотеза H_0 , в то время, как она неверна.

	Основная гипотеза верна	Основная гипотеза неверна
Приняли основную гипотезу	Верное решение	Ошибка II рода
Отклонили основную гипотезу	Ошибка I рода	Верное решение

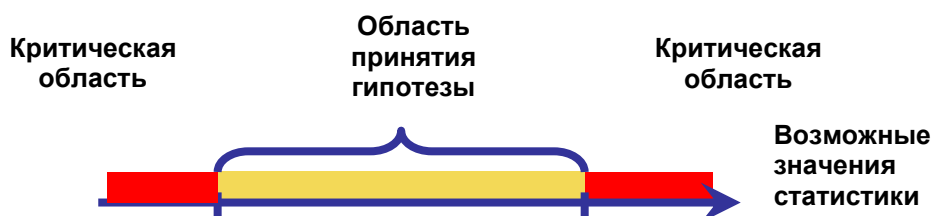
Уровнем значимости гипотезы называют допустимую вероятность совершить ошибку первого рода, то есть принять неверную гипотезу. Обозначается α .

Уровень значимости α выбирается исследователем до того, как будет проверяться гипотеза. Значение обычно выбирается небольшим, например, 0,1; 0,05 или 0,01.

Каким образом на основании выборки принимается решение? Для этого необходима специальная функция, называемая **статистикой** или **критерием**. Эта функция зависит от выборки и потому является случайной функцией.

Множество значений статистики включает:

- **область принятия гипотезы**, то есть множество тех значений статистики, при которых гипотеза H_0 принимается
- **критическую область**, то есть множество тех значений статистики, при которых гипотеза H_0 отклоняется и принимается альтернативная гипотеза:



6.3. Одномерные статистические тесты

Пример 6.1:

1. Задаем имена сотрудников фирмы:

```
> names<-c("Коля", "Женя", "Петя", "Саша", "Катя", "Вася", "Жора")
```

2. Ваша задача получить что-то такое:

	names	weight	height	sex	salary
1	Коля	69	180	Male	21
2	Женя	68	165	Femail	19
3	Петя	93	175	Male	27
4	Саша	87	178	Male	11
5	Катя	59	177	Femail	102
6	Вася	82	176	Male	25
7	Жора	72	170	Male	21

3. Начнём с одномерных тестов, которые позволяют проверить утверждения относительно того, как распределены исходные данные:

```
> t.test(salary,mu=32)
```

```
One Sample t-test
```

```
data: salary
t = 0.0243, df = 6, p-value = 0.9814
alternative hypothesis: true mean is not equal to 32
95 percent confidence interval:
 3.468127 61.103302
sample estimates:
mean of x
32.28571
```

Наша нулевая гипотеза состояла в том, что средняя ЗП=32. Для проверки был использован тест Стьюдента для одномерных данных. (на самом деле Student – это псевдоним Уильяма Госсета).

Тест базируется на идее, что выборка имеет нормальное распределение! И при этом крайне рекомендуется избавиться от выбросов.

Статистические тесты пытаются высчитать, так называемую, тестовую статистику. Затем на основании этой статистики рассчитывается р-величина или p-value, отражающая вероятность ошибки первого рода (ее ещё называют «ложной тревогой»). Принято считать, что нулевой гипотезе соответствует ситуация «по умолчанию». Вычисленная р-величина используется для сравнения с заранее заданным порогом (уровнем) значимости. Если р-величина ниже выбранного порога уровня значимости (как правило 0,05), то нулевая гипотеза отвергается, а если выше, то принимается. В этом примере гипотеза принимается.

Функция возвращает список со следующими компонентами:

- data.name – имена переменных, содержащих выборки, которые были переданы функции t.test в качестве аргументов x и y;
- method – строка с названием теста;
- statistic – значение статистики теста, которую принято обозначать буквой t ;
- parameter df – количество степеней свободы t -статистики;
- p.value – p-value для полученного значения статистики;
- alternative – строка с описанием альтернативной гипотезы;
- conf.int – доверительный интервал;

- estimate – оцененные по выборкам значения мат. ожиданий;
- null.value – значение параметра mu, использовавшееся для теста.

Перейдём к анализу вывода функции. Вычисляемая статистика в нашем случае t (критерий Стьюдента). При шести степенях свободы (df=6, поскольку у нас всего 7 наблюдений - людей) тест даёт нам величину p.value очень близкую к единице (0.9814 ~ 1). Какой бы порог мы не приняли (0.1%, 1% или 5%), это значение всё равно больше. Следовательно, мы придерживаемся нулевой гипотезы (наша информация о средней зарплате скорее верна, чем нет). Кроме того, функция выдаёт ещё и доверительный интервал (confidence interval), в котором, по её «мнению», может находиться «истинное» среднее. В данном случае он очень широк — от трёх с половиной тысяч до 61 тысячи рублей (это всё из-за высокой Катиной зарплаты).

Существует так же непараметрический **аналог** этого теста, то есть теста, **не связанного предположениями о нормальном распределении**. Это ранговый тест Уилкоксона (Wilcoxon signed-rank test).

4. Для чистоты эксперимента возьмем заведомо известные и одинаковые данные:

Пример 6.2:

```
> numb=rnorm(100)
> plot(1:100, numb)
```

```
> summary(numb)
   Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
-2.14800 -0.60540  0.06831  0.04330  0.70520  2.27700
```

5. Выведите гистограмму этого распределения и рядом сглаженную гистограмму

6. Проведем тест Стьюдента над этой выборкой:

```
t.test(numb,mu=mean(numb),conf.int=TRUE)
```

```
One Sample t-test
```

```
data:  numb
t = 0, df = 99, p-value = 1
alternative hypothesis: true mean is not equal to 0.04329895
95 percent confidence interval:
 -0.1504809  0.2370788
sample estimates:
mean of x
0.04329895
```

7. Проанализируйте тест Стьюдента.

8. Проведем тест Уилкоксона над этой же выборкой:

```
wilcox.test(numb,mu=mean(numb),conf.int=TRUE)
```

```
wilcoxon signed rank test with continuity correction
```

```
data:  numb
V = 2551, p-value = 0.9301
alternative hypothesis: true location is not equal to 0.04329895
95 percent confidence interval:
 -0.1496288  0.2550488
sample estimates:
(pseudo)median
0.05460622
```

9. Проанализируйте тест Уилкоксона, сравните результаты.

Эта функция выводит практически то же самое. Обратите внимание, что тест связан не со средним, а с **медианой**. Соответственно, вычисляется (если задать `conf. int=TRUE`) доверительный интервал. Здесь он значительно уже.

6.4. Проверка на нормальность распределения

Некоторые статистические методы (например, ANOVA или дисперсионный анализ) основаны на том, что данные имеют нормальное распределение. Поэтому вопрос соответствует ли распределение данных нормальному или хотя бы напоминает нормальное является очень и очень важным. В R реализовано несколько разных техник, отвечающих на вопрос о нормальности. Во-первых, это статистические тесты. Самый простой из них — тест Шапиро-Уилкса (Shapiro-Wilk test):

```
> shapiro.test(salary)
```

```
Shapiro-wilk normality test

data: salary
W = 0.6116, p-value = 0.0003726
```

```
> shapiro.test(numb)
```

```
Shapiro-wilk normality test

data: numb
W = 0.9914, p-value = 0.7788
```

Для выполнения теста Шапиро-Уилкса предназначена функция `shapiro.test(x)`, принимающая на вход выборку `x` объема не меньше 3 и не больше 5000. Функция возвращает список со следующими компонентами:

- `method` – строка с названием теста;
- `statistic` – значение статистики теста, которую принято обозначать буквой *W*;
- `data.name` – имя переменной, содержащей выборку, которая была передана функции `shapiro.test` в качестве аргумента.
- `p.value` – аппроксимация `p-value` для полученного значения статистики;

Следует отметить, что, несмотря на то, что возвращаемое значение является списком, его вывод на экран обрабатывается особым образом, позволяя более компактно представить данные о выполненном тесте.

Таким образом, на основании теста Шапиро-Уилкса можно заключить, что распределение данных в `salary` существенно отличается от нормального, тогда как анализ распределения `numb` (заведомо нормального) дало подтверждение гипотезы.

Рассмотрим еще несколько примеров. Применим тест Шапиро-Уилкса к выборкам из различных распределений, сгенерированных с помощью стандартных функций языка R.

```
1 > set.seed(0)
2 > shapiro.test(rnorm(100, mean = 2, sd = 5))
3
4 Shapiro-Wilk normality test
5 data: rnorm(100, mean = 2, sd = 5)
6 W = 0.9896, p-value = 0.6303
7
8
```

```

> set.seed(0)
> shapiro.test(runif(100, min = -10, max = 10))
11
12      Shapiro - Wilk normality test
13
14 data:      runif(100, min = -10, max = 10)
15 W = 0.9561, p-value = 0.002126

```

В данном случае при уровне значимости, например, $\alpha = 0.05$ для выборки, сгенерированной функцией `runif`, гипотеза H_0 должна быть принята (так как $p\text{-value} > \alpha$), а для выборки, полученной с помощью `runif()`, H_0 следует отклонить, приняв альтернативную гипотезу.

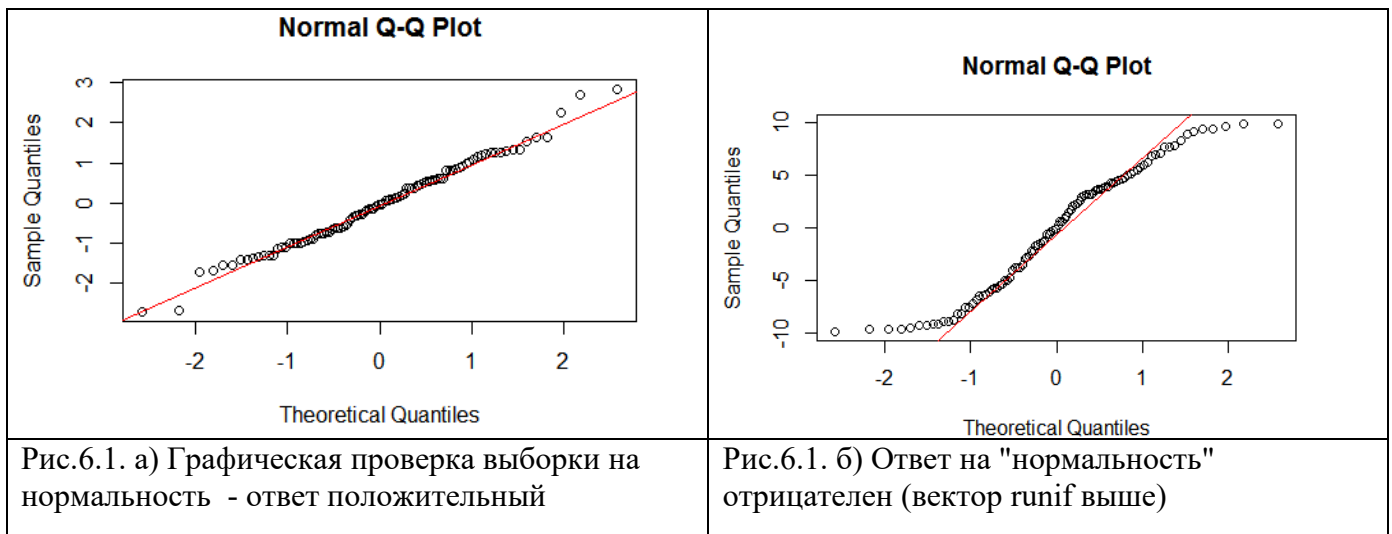
Напомним, что `rmnorm()` генерирует столько случайных чисел, распределённых по нормальному закону, сколько указано в его аргументе. Это аналог функции `sample()`. Для того чтобы результаты при вторичном воспроизведении были теми же, использована функция `set.seed()`. Эта функция подстраивает встроенный в R генератор случайных чисел так, чтобы числа в следующей команде были сгенерированы по одному и тому же «закону». Кстати говоря, генераторов случайных чисел в R целых шесть (см. `help(set.seed)`)!

Другой популярный способ проверить, насколько распределение похоже на нормальное — графический (см. рис. 6.1 и 6.2). Это делается так:

```

> qqnorm(numb)
> qqline(numb, col=2)

```

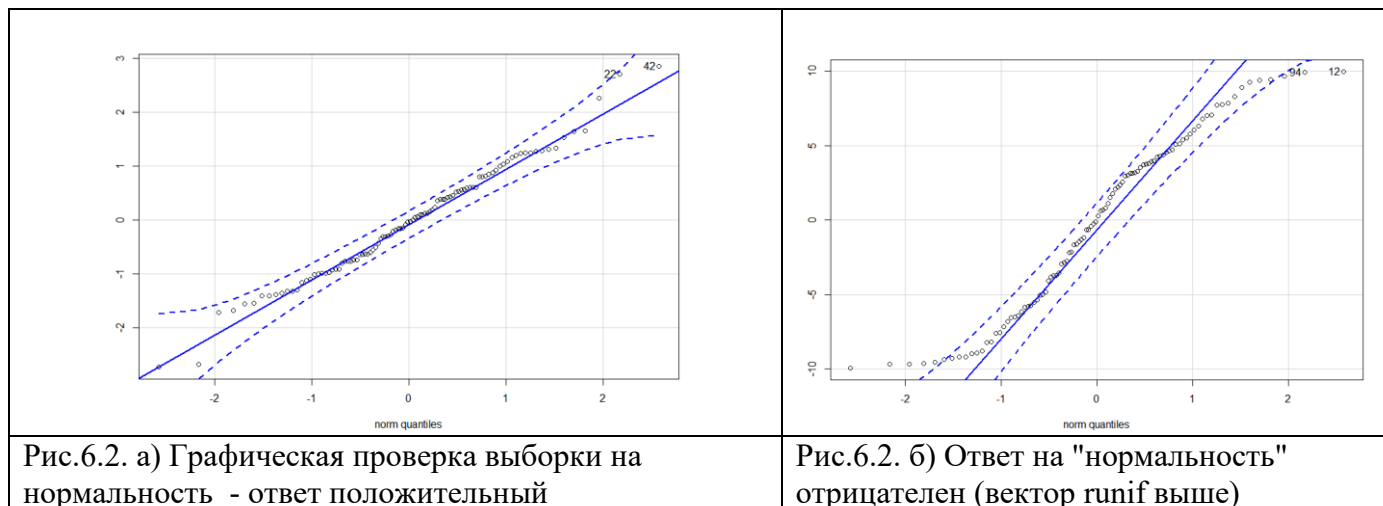


Стандартные `qqplot()` функции в R не обеспечивают доверительные интервалы, но `qqPlot()` функция из пакета `car` показывает и их.

```

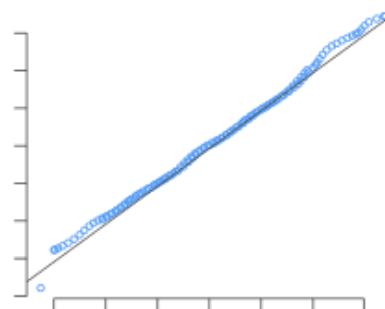
install.packages('car')
library(car)
qqPlot(numb)
[1] 42 22

```

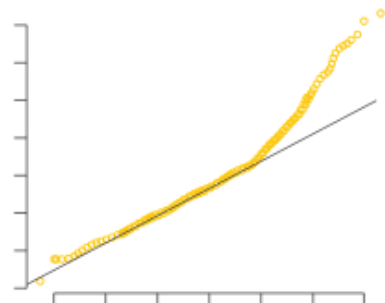
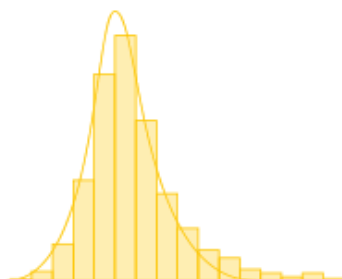


Квантильно-квантильный график (график QQ) показывает распределение данных относительно ожидаемого нормального распределения и трактуется от так:

Normally distributed data



Right-skewed data



Left-skewed data

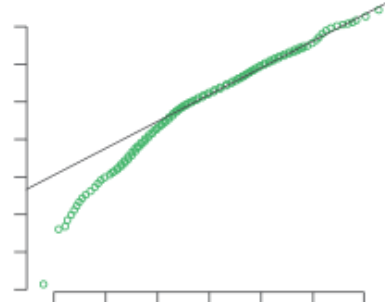
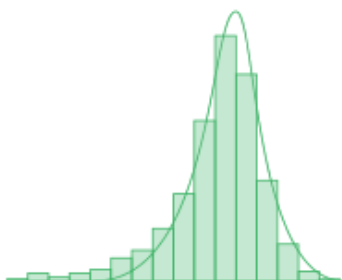


Рис. 6.3. Квантильно-квантильный график

6.5. Сравнение двух независимых выборок

При сравнении двух выборок проверяемая нулевая гипотеза состоит в том, что обе эти выборки происходят из нормально распределенных генеральных совокупностей с одинаковыми средними значениями:

$$H_0 : \mu_1 = \mu_2$$

Поскольку эти генеральные средние мы оцениваем при помощи выборочных средних значений, формула t -критерия приобретает вид

$$t = \frac{\bar{x}_1 - \bar{x}_2}{S_{\bar{x}_1 - \bar{x}_2}}$$

В знаменателе приведенной формулы находится стандартная ошибка разницы между выборочными средними, которая в общем виде рассчитывается как

$$S_{\bar{x}_1 - \bar{x}_2} = \sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}},$$

где s_1^2 и s_2^2 - выборочные оценки дисперсии. При соблюдении условия о равенстве групповых дисперсий приведенная формула приобретает более простой вид (подробнее см. [здесь](#)). Интерпретация t -критерия, рассчитанного для двух выборок, выполняется точно так же, как и в случае с одной выборкой (см. выше).

Рассмотрим пример о суточном расходе энергии (expend) у худощавых женщин (lean) и женщин с избыточным весом (obese), приведенный в книге Питера Дальгаарда (*Dalgaard P (2008) [Introductory statistics with R](#). Springer*). Данные из этого примера (подробнее см. `?energy`) входят в состав пакета ISwR, сопровождающего книгу (если он у Вас не установлен, выполните команду `install.packages("ISwR")`):

```
library(ISwR)
data(energy)
attach(energy)
energy
  expend stature
1     9.21   obese
2     6.53    lean
3     6.48    lean
4     8.08    lean
5     8.09    lean
6    10.15    lean
7     8.40    lean
8    10.88    lean
9     6.13    lean
10    6.90    lean
11    11.51   obese
12    12.79   obese
13     6.05    lean
14    11.85   obese
15     9.97   obese
16     6.48    lean
17     8.79   obese
18     9.69   obese
19     9.68   obese
```

Соответствующие средние значения потребления энергии в рассматриваемых группах пациенток составляют (подробнее о примененной ниже функции `tapply()` см. [здесь](#)):

```
tapply(expend, stature, mean)  
lean obese  
8.07 10.30
```

Напомню, что прежде, чем сравнить потребление энергии в разных группах для двухвыборочного теста, необходимо проверить а) нормальность распределения и б) равенство дисперсий:

а) Тест на нормальность распределения:
Тест Шапиро.

б) Тест на равенство дисперсий для полных и худых женщин. Гипотеза H_0 – дисперсии равны – принимается за основную:

```
bartlett.test(expend~stature, data=energy)  
  
Bartlett test of homogeneity of variances  
data: expend by stature  
Bartlett's K-squared = 0.13624, df = 1, p-value = 0.712
```

И, наконец, вопрос: различаются ли эти средние значения статистически, проверим на основе гипотезы об отсутствии разницы при помощи t -теста:

```
t.test(expend ~ stature)  
  
Welch Two Sample t-test  
  
data: expend by stature  
t = -3.8555, df = 15.919, p-value = 0.001411  
alternative hypothesis: true difference in means is not equal to 0  
95 percent confidence interval:  
-3.459167 -1.004081  
sample estimates:  
mean in group lean mean in group obese  
8.066154 10.297778
```

Обратите внимание на использование знака `~` в вызове функции `t.test()` – это стандартный для R способ записи *формул*, описывающих связь между переменными. В нашем случае выражение `expend ~ stature` можно расшифровать как "зависимость суточного потребления энергии (`expend`) от статуса пациентки (`stature`)".

Согласно полученному значению P ($p\text{-value} = 0.001411$), средние значения потребления энергии у женщин из рассматриваемых весовых групп статистически значимо различаются. Отвергая нулевую гипотезу о равенстве этих средних значений, мы рискуем ошибиться с вероятностью лишь около 0.1%. При этом истинная разница между средними значениями с вероятностью 95% находится в диапазоне от -3.5 до -1.0 (см. 95 percent confidence interval).

Следует подчеркнуть, что при выполнении двухвыборочного t -теста R по умолчанию принимает, что дисперсии сравниваемых совокупностей *не равны*, и, как следствие, выполняет t -тест в модификации Уэлча (подробнее см. [здесь](#)). Мы можем изменить такое поведение программы, воспользовавшись аргументом `var.equal = TRUE`: (от *variance* - дисперсия, и *equal* - равный):

```
t.test(expend ~ stature, paired = FALSE, var.equal = TRUE)  
  
Two Sample t-test  
  
data: expend by stature
```

```

t = -3.9456, df = 20, p-value = 0.000799
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -3.411451 -1.051796
sample estimates:
mean in group lean mean in group obese
 8.066154          10.297778

```

Р-значение стало еще меньше, и мы так же, как и после теста в модификации Уэлча, можем сделать вывод о наличии существенной разницы между средними. Однако такое совпадение выводов будет иметь место не всегда и, следовательно, на разницу между групповыми дисперсиями (или ее отсутствие) следует обращать серьезное внимание при выборе и интерпретации того или иного варианта t-теста.

6.6. Задания к лабораторной работе

Набор данных для исследования - Athlen_Events (размещен на Яндекс диске) – это исторический набор данных о современных Олимпийских играх, включая все игры от Афин 1896 года до Рио-2016. Эти данные собраны из www.sports-reference.com в мае 2018 года.

Вид спорта брать по варианту из таблицы.

1. Провести дескриптивный анализ данных (EDA).
2. Провести проверку на нормальность и дисперсию. Из чего сделать вывод о требуемом тесте.
3. Проверьте гипотезу о среднем весе спортсменов выбранного вида спорта - одновыборочный критерий(из таблицы вариантов).
4. Проверьте гипотезу о равенстве среднего веса женщин (мужчин) в двух разных выбранных видах спорта (сравнение двух независимых выборок – двухвыборочный критерий.).

Варианты:

№	ФИО	Вар №	№	ФИО	Вар №
1.	Блохин Михаил Максимович	Плавание	1.	Артамонов Владимир Александрович	Конный спорт
2.	Бусыгин Андрей Алексеевич	велоспорт	2.	Асатрян Тигран Рубикович	пятиборье
3.	Вишняков Даниил Иванович	Фристайл	3.	Ахвердян Гурген Артурович	Стрельба из лука
4.	Власов Сергей Михайлович	Легкая атлетика	4.	Бабаянц Эдуард Завенович	Греко-римская борьба
5.	Водопьянов Александр Сергеевич	Фигурное катание	5.	Гладкова Марина Алексеевна	Теннис
6.	Дударев Всеволод Александрович	Гтяжелая атлетика	6.	Диденко Артемий Анатольевич	Водное поло
7.	Едиджи Эдуард Асланович	Футбол	7.	Карпенко Владислава Андреевна	Фигурное катание
8.	Ерыкалов Арсений Сергеевич	Баскетбол	8.	Книхута Максим Игоревич	Футбол
9.	Жадько Ксения Антоновна	Гимнастика	9.	Котов Данил Евгеньевич	Баскетбол
10.	Забелин Ярослав Сергеевич	Волейбол	10.	Ксенофонтов Богдан Васильевич	Легкая атлетика
11.	Захарян Давид Даниэльевич	Хоккей с мячом	11.	Лигостаев Артём Алексеевич	Хоккей
12.	Макаренко Даниил Геннадьевич	Конный спорт	12.	Мержоев Умар Русланович	гимнастика
13.	Русинов Максим Евгеньевич	Гроеборье	13.	Назаренко Георгий Сергеевич	Фристайл
14.	Сейтмамутов Эрнест Русланович	Греко-римская борьба	14.	Сластенов Константин Вячеславович	Волейбол
15.	Тузовский Евгений Евгеньевич	Горнолыжный спорт	15.	Таширов Никита Олегович	Плавание
16.	Щербаков Максим Игоревич	Борьба	16.	Трофименко Мария Алексеевна	Бадминтон
17.	Придава Александр	Велоспорт			

18.			17.	Фадеева Виолетта Александровна	Худ. гимнастика
19.	Баландин Алексей Сергеевич	Стрельба из лука	18.	Ананко Иван	волейбол
20.	Галуза Виолетта Владимировна	Водное поло	19.	Крапоткин Максим	футбол
21.	Долгов Михаил Игоревич	Конькобежный спорт	20.	Волков	Лыжный спорт
22.	Евсеев Богдан Владиславович	Легкая атлетика	21.	Дерябин	Тяжелая атлетика
23.	Залевский Тимофей Олегович	Стрельба из лука	22.		28
24.	Зинченко Диана Романовна	Гимнастика	23.		17
25.	Мишенькина Валерия Андреевна	Фигурное катание	24.		27
26.	Кесидис Константинос	Тяжелая атлетика	25.		16
27.	Костров Александр Алексеевич	Футбол	26.		22
28.	Лебедев Никита Витальевич	Баскетбол	27.		23
29.	Салтыкова Дарья Алексеевна	Волейбол	28.		24
30.	Тертышный Дмитрий Сергеев	Каратэ	29.		
31.	Хить Анна Александровна	Теннис	30.		
32.	Хромушина Варвара Геннадьевна	Плавание	31.		
33.	Челепис Таисия Геннадьевна	Фристайл	32.		
34.	Щеглов Дмитрий Викторович	Бокс			

Приложение А

Справочник по функциям:

Получение информации о функциях и объектах R

Справочная информация по большинству функций R доступна онлайн (например, на сайте [Факультета статистики Калифорнийского университета в Беркли](#)). Непосредственно из командной строки информацию о функциях и объектах R можно получить при помощи следующих команд:

help (тема)

```
# документация по "теме"; например команда help(plot) выдаст страницу
# со справочной информацией по функции plot()
```

```
?тема
```

```
# это эквивалент команды help(тема)
```

help.search ("тема")

```
# поиск информации по "теме" во всей встроенной справочной системе R;
# обратите внимание на необходимость кавычек
```

apropos ("выражение")

```
# выполняет поиск всех функций, в имени которых содержится "выражение";
# обратите внимание на необходимость кавычек
```

help.start ()

```
# локально запускает html-версию встроенной справочной системы R
```

str (a)

```
# отображает внутреннюю структуру объекта a (от "structure" - структура)
```

summary (a)

```
# выводит обобщенную информацию об объекте a;
# как правило, это набор статистических параметров, описывающих a,
```

```
# однако summary() является функцией общего назначения
# и конкретный результат ее выполнения будет зависеть от класса объекта a
```

ls()

```
# выводит список всех объектов, находящихся в рабочей среде программы;
# для поиска объектов, в именах которых имеется повторяющийся элемент,
# используйте дополнительный аргумент pat = "паттерн",
# например, ls(pat = "mean")
```

dir()

```
# отображает содержимое рабочей директории программы
```

methods(a)

```
# выводит список методов, применимых к объекту S3-класса a
```

methods(class = class(a))

```
# выводит список методов, применимых к объекту класса a
```

Ввод и сохранение данных

data(x)

```
# загружает таблицу данных x,
# сохраненную ранее при помощи команды save()
```

library(x)

```
# загружает дополнительный пакет x ("библиотеку"), содержащий набор функций
# для выполнения анализа определенного вида; помимо функций, расширяющих
# базовые возможности R, пакеты могут содержать также таблицы с данными
```

read.table(file)

```
# считывает файл с данными и создает из него таблицу данных (dataframe);
# по умолчанию предполагается, что значения переменных в считываемом файле
# разделены пробелом (аргумент sep = " "; от слова separator - разделитель),
# но, например, в csv-файлах разделителем служит запятая (sep = ",");
# для превращения первой строки считываемого файла в заголовки столбцов,
# необходимо включить аргумент header (header = TRUE);
# аргумент skip = n можно использовать для того, чтобы пропустить
# первые n строк файла при считывании;
# о других подробностях работы с этой функцией можно узнать,
# выполнив команду help(read.table)
```

read.csv("filename", header = TRUE)

```
# то же, что и read.table(),
# но с готовыми настройками для считывания csv-файлов
```

read.delim("filename", header = TRUE)

```
# то же, что и read.table(),
# но с готовыми настройками для считывания файлов, в которых
# значения переменных разделены знаком табуляции (tab-delimited files)
```

save(file, ...)

```
# сохраняет указанные объекты (...) в двоичном файле
# XDR-формата, с которым можно работать в любой операционной системе
```

save.image(file)

```
# сохраняет все объекты, созданные в ходе работы,
# в виде специфичного для R rda-файла
```

cat(..., file = "", sep = " ")

```
# превращает все указанные аргументы (...)
# в текст и сохраняет результат в виде файла; разделитель между
# получаемыми текстовыми значениями задается при помощи аргумента sep
```

print(a, ...)

```
# выводит на экран объект a; это функция общего назначения -
# конкретный результат ее работы будет зависеть от класса объекта a
```

format(x,...)

позволяет отформатировать объект x так, что
он будет выглядеть "аккуратно" при выводе на экран

write.table(x, file = "", row.names = TRUE, col.names = TRUE, sep = " ")

сохраняет объект x в файл, предварительно преобразовав этот объект
в таблицу данных (dataframe); если включен аргумент quote (quote = TRUE),
текстовые значения и значения уровней факторов будут заключены в
двойные кавычки ("); аргумент sep позволяет задать разделитель между
значениями переменных в таблице; используйте col.names = NA для создания
таблицы, с которой в дальнейшем предполагается работать в программах
вроде Excel – так заголовки столбцов таблицы не окажутся смещенными

sink(file)

выводит результаты выполнения других команд в файл в режиме
реального времени; для прекращения действия этой команды выполните
команду sink() (именно так – ничего не указывая в скобах)

Работая под операционной системой Windows, можно скопировать таблицу
с данными из Excel в буфер памяти и сохранить ее в виде
R-объекта следующим образом:

read.delim("clipboard")

Чтобы скопировать таблицу данных, хранящуюся в R-объекте x, в буфер памяти
для последующей вставки в Excel, можно выполнить следующую команду:

write.table(x, "clipboard", sep = "\t", col.names = NA)

Для операций с данными, хранящимися в базах данных, существуют специальные
R-библиотеки: RODBC, DBI, RMySQL, RPgSQL, ROracle. Библиотеки XML, hdf5,
netCDF могут пригодиться для работы с файлами других форматов

Создание векторов и таблиц с данными

c(...)

функция общего назначения (от concatenation – объединение, слияние);
объединяет аргументы в один вектор определенного типа;
например, команда:

a = c(1, 2, 3)

приведет к созданию числового вектора, содержащего значения от 1 до 3;
эквивалентом этой команды является

a = **c**(1:3)

seq(from = , to = , by)

формирует последовательность числовых или других значений
с заданным шагом; например, команда

a = **seq**(from = 1, to = 3, by = 0.5) # или просто a = seq(1, 3, 0.5)

приведет к созданию числового вектора a, содержащего значения
1.0, 1.5, 2.0, 2.5, 3.0

rep(x, times =)

повторяет x то количество раз, которое задано аргументом times;
например, команда

a = **rep**(5, 3)

приведет к созданию вектора a, содержащего значения 5, 5, 5;
используя аргумент each, можно повторить каждый элемент объекта x
необходимое количество раз; так, результатом выполнения команды

rep(c(1,2,3), each = 2)

будет 1 1 2 2 3 3

data.frame(...)

создает таблицу данных из поименованных или непоименованных аргументов;
например, выполнение команды

a = **data.frame**(Number = 1:4, Letter = **c**("a","B","c","d"))

```

# приведет к созданию таблицы с двумя столбцами -
# Number (содержит числа от 1 до 4) и Letter (содержит буквы a, B, c, d)

list(...)
# создает список объектов; в отличие от столбцов таблицы данных,
# список может содержать объекты разной длины, например:
list(a = c(1,2), b = "hi", c = TRUE)

array(x, dim = )
# создаст массив данных из объекта x; аргумент dim используется для указания
# размерности массива (например, dim = c(3,4,2));
# если объект x не обладает достаточной длиной, его элементы
# при создании массива будут использованы повторно

matrix(x, nrow = , ncol = )
# создает матрицу из вектора x; количество строк задается аргументом nrow,
# количество столбцов - аргументом ncol;
# если объект x не обладает достаточной длиной,
# его элементы при создании матрицы будут использованы повторно ("recycling")

factor(x, levels = )
# превращает вектор x в фактор; число уровней фактора задается при помощи
# аргумента levels

gl(n, k, length = n*k, labels = 1:n)
# еще одна команда для создания факторов; n - количество уровней фактора;
# k - число повторов для каждого уровня; length - размер итогового объекта;
# labels - необязательный аргумент, который можно использовать для указания
# названий каждого уровня фактора; например, выполнение команды
a = gl(2, 8, labels = c("Control", "Treatment"))
# приведет к созданию вектора a, являющегося фактором с двумя уровнями -
# Control и Treatment; каждая из меток "Control" и "Treatment"
# будет повторена 8 раз

rbind(...)
# эта функция построчно объединяет свои аргументы,
# создавая матрицу или таблицу данных

cbind(...)
# функция, аналогичная предыдущей; отличие состоит лишь в том, что cbind()
# объединяет свои аргументы в одну матрицу или таблицу данных по столбцам,
# а не по строкам

```

Индексирование векторов

```

x[n]
# возвращает n-й элемент вектора x

x[-n]
# возвращает все элементы вектора x, за исключением n-го

x[1:n]
# возвращает первые n элементов вектора x

x[-(1:n)]
# возвращает элементы вектора x с n+1 до последнего

x[c(1,4,2)]
# возвращает определенные элементы вектора x (здесь 1-й, 4-й и 2-й)

x["name"]
# возвращает элемент вектора x с именем "name"

x[x > 3]

```

```
# возвращает все элементы вектора x со значением >3

x[x > 3 & x < 5]
# возвращает все элементы вектора x со значением >3 и <5

x[x %in% c("a", "and", "the")]
# возвращает только те элементы вектора x, которые указаны после
# оператора %in% (здесь из вектора x были бы извлечены текстовые значения
# "a", "and", и "the")
```

Индексирование списков, матриц и таблиц данных

```
x[[n]]
# возвращает n-й элемент списка x

x[["name"]]
# возвращает элемент списка или таблицы x с именем "name"

x$name
# команда, идентичная предыдущей
```

Конвертация R-объектов

```
# Следующие команды конвертируют объект x в объект соответствующего класса:
as.array(x) # массив данных

as.data.frame(x) # таблица данных

as.numeric(x) # числовой вектор

as.logical(x) # логический вектор

as.character(x) # текстовый вектор
```

Получение информации о переменной

```
# Следующие команды позволяют проверить, принадлежит ли объект x или его
# отдельные элементы к определенному типу:

is.na(x) # отсутствующее значение

is.null(x) # ноль

is.array(x) # массив данных

is.data.frame(x) # таблица данных

is.numeric(x) # числовой вектор

is.character(x) # текстовый вектор

# используйте команду method(is) для просмотра полного списка
# аналогичных функций

length(x)
# возвращает число элементов, содержащихся в объекте x

dim(x)
# возвращает размерность объекта x; используя команду вроде
dim(x) = c(3, 2)
# можно принудительно задать определенную размерность объекта x

dimnames(x)
# возвращает или присваивает имена размерностей объекта x
```

nrow(x)

возвращает число строк в таблице или матрице x

ncol(x)

то же для столбцов

class(x)

возвращает или задает (например, `class(x) = "myclass"`) класс объекта x

unclass(x)

удаляет атрибут класса у объекта x

attr(x, **which**)

возвращает или задает атрибут "which" объекта x

attributes(obj)

возвращает или задает список атрибутов объекта obj

Извлечение данных и манипуляции с ними

which.max(x)

возвращает порядковый номер элемента объекта x с максимальным значением

which.min(x)

возвращает порядковый номер элемента объекта x с минимальным значением

rev(x)

меняет порядок элементов объекта x на обратный

sort(x)

сортирует элементы объекта x по возрастанию;

для сортировки по убыванию используйте

rev(**sort**(x))

cut(x, breaks)

делит вектор x на равные интервалы (факторы); в качестве аргумента breaks

может выступать либо необходимое число интервалов, либо вектор,

содержащий перечень "точек разрыва"

match(x, y)

ищет, какие значения объекта y совпадают со значениями объекта x

и возвращает порядковые номера первых из совпадающих значений;

например, если

y = **c**("a", "b", "c", "d", "e", "f")

x = **c**("d", "f") # то результатом команды `match(x, y)` будет вектор

[1] 4 6

which(x == a)

проверяет, какие элементы объекта x равны a, и возвращает вектор,

содержащий порядковые номера этих элементов

na.omit(x)

исключает отсутствующие значения из объекта x;

если x является матрицей или таблицей данных, то исключается

каждая строка, содержащая хотя бы одно отсутствующее значение

na.fail(x)

возвращает сообщение об ошибке если объект x содержит хотя бы

одно отсутствующее значение

unique(x)

если x – вектор или таблица данных, эта команда создаст

соответствующий объект, в котором не будет повторяющихся значений

table(x)

возвращает таблицу с частотами встречаемости каждого

```

# значения x

subset(x, ...)
# отфильтровывает и возвращает ту часть объекта x, которая
# соответствует определенному условию; например, команда
a = subset(x < 5)
# создаст вектор a, который будет содержать только те значения x,
# которые не превышают 5; если x – таблица данных, то
# можно использовать аргумент select для указания столбцов,
# которые необходимо извлечь из этой таблицы

sample(x, size)
# производит (квази)случайный отбор элементов из объекта x
# в количестве size; аргумент replace = TRUE позволяет
# осуществлять случайный отбор элементов с их возвратом
# в исходную совокупность x

prop.table(x, margin=)
# рассчитывает маргинальные частоты таблиц; чтобы понять, как
# работает данная функция, выполните следующие команды:
m = matrix(1:4, 2)
m
prop.table(m, margin = 1)

apply(x, MARGIN, FUN = ...)
# возвращает вектор, массив или список значений, полученных путем
# применения функции FUN к определенным элементам массива или матрицы x;
# подлежащие обработке элементы x указываются при помощи аргумента
# MARGIN; например, если x – матрица, то при MARGIN = 1 функция FUN
# будет применена к каждой строке матрицы, при MARGIN = 2 –
# к каждому столбцу матрицы

lapply(x, FUN = ...)
# возвращает список той же длины, что и x; при этом значения в
# новом списке будут результатом применения функции FUN к элементам
# исходного объекта x

tapply(x, INDEX, FUN = ...)
# применяет функцию FUN к каждой совокупности значений x,
# созданной в соответствии с уровнями определенного фактора;
# перечень факторов указывается при помощи аргумента INDEX

by(data, INDICES, FUN = ...)
# аналог tapply(), применяемый к таблицам данных

merge(a, b)
# объединяет две таблицы данных (a и b) по общим столбцам или строкам

aggregate(x, by = ..., FUN = ...)
# разбивает таблицу данных x на отдельные наборы данных, применяет
# к этим наборам определенную функцию FUN и возвращает результат
# в удобном для чтения формате; аргумент by задает список группирующих
# элементов (например, уровней факторов)

stack(x)
# преобразует данные, представленные в объекте x в виде отдельных
# столбцов, в таблицу данных (если x – список, то результатом
# будет один единственный столбец со всеми элементами x)

unstack(x)
# выполняет операцию, обратную действию функции stack()

reshape(x)
# преобразует таблицу данных из "широкого формата" (повторные измерения
# какой-либо величины записаны в отдельных столбцах таблицы) в таблицу
# "узкого формата" (повторные измерения идут одно под одним в пределах

```

одного столбца)

paste(..., sep = ...)

конвертирует векторы в текстовые переменные и объединяет их
в одно текстовое выражение; аргумент sep позволяет задать
текстовое выражение, которое будет разделять значения объединяемых
векторов (по умолчанию это пробел)

substr(x, start, stop)

извлекает определенную часть из текстового вектора x; аргументы
start и stop служат для указания позиции первого и последнего
элементов извлекаемой части вектора x

strsplit(x, split)

разбивает текстовый вектор x в соответствии с паттерном,
заданным при помощи аргумента split

grep(pattern, x)

производит поиск частей текстового вектора x, которые
совпадают с паттерном, указанным при помощи аргумента pattern;
для обзора других функций, предназначенных для работы с регулярными
выражениями, см. ?regex

gsub(pattern, replacement, x)

заменяет все части текстового вектора x, соответствующие паттерну
pattern, на выражение, заданное при помощи аргумента replacement

tolower(x)

преобразует все буквы текстового вектора x в "маленькие"

toupper(x)

преобразует все буквы текстового вектора x в "большие"

match(x, table)

выполняет поиск элементов в векторе table, которые совпадают
со значениями из вектора x, и возвращает порядковые номера первых
таких совпадений

x %in% table

команда, аналогичная предыдущей

pmatch(x, table)

выполняет поиск элементов в векторе table, которые частично совпадают
с элементами вектора x, и возвращает порядковые номера первых
таких совпадений

nchar(x) # возвращает количество знаков в текстовом векторе x

Математические функции

Ниже приведен ряд математических функций, действие которых
должно быть понятно из их названий:

sin(x); cos(x); tan(x); asin(x); acos(x); atan(x);
atan2(x); log(x); log10(x); exp(x)

max(x)

максимальное значение в числовом векторе x

min(x)

минимальное значение в числовом векторе x

range(x)

возвращает минимальное и максимальное значения из числового
вектора x, т.е. выполняет команду с(min(x), max(x))

sum(x)

```

# сумма всех элементов x

prod(x)
# произведение всех элементов x

mean(x)
# арифметическая средняя совокупности x

median(x)
# медиана совокупности x

quantile(x, probs = ...)
# рассчитывает выборочные квантили, соответствующие определенным
# вероятностям (по умолчанию это 0, 0.25, 0.5, 0.75, 1); при помощи
# аргумента probs можно задать вектор с любыми интересующими
# пользователя вероятностями

weighted.mean(x, w)
# средневзвешенная средняя по вектору x; аргумент w служит для
# указания весов

rank(x)
# ранжирует элементы x

var(x)
# дисперсия совокупности x (рассчитывается для  $n - 1$ )

cov(x)
# то же, что и var(); если x - матрица или таблица данных,
# рассчитывается ковариационная матрица

sd(x)
# стандартное отклонение совокупности x

cor(x)
# возвращает корреляционную матрицу если x является матрицей или
# таблицей данных (результатом будет 1 если x является вектором)

var(x, y); cov(x, y)
# возвращает ковариации между x и y, или между всеми столбцами x и y
# (в случае, если x и y являются матрицами или таблицами данных)

cor(x, y)
# возвращает параметрический коэффициент корреляции Пирсона
# (или корреляционную матрицу, если x и y являются матрицами
# или таблицами данных)

round(x, n)
# округляет x до n знаков после запятой

log(x, base)
# рассчитывает логарифм x по основанию base

scale(x)
# если x является матрицей, нормализует значения каждого столбца
# (т.е. вычитает от каждого значения среднюю по столбцу и делит результат
# на стандартное отклонение по этому столбцу)

pmin(x, y)
# возвращает вектор с минимальными значениями из каждой пары x[i], y[i]

pmax(x, y)
# команда, идентичная предыдущей, но для максимальных значений

cumsum(x)
# возвращает вектор с кумулятивными суммами по вектору x

```

cumprod(x)

возвращает вектор с кумулятивными произведениями по вектору x

cummin(x)

возвращает вектор с кумулятивными минимумами по вектору x

cummax(x)

возвращает вектор с кумулятивными максимумами по вектору x

union(x, y)

объединяет элементы векторов x и y; результирующий вектор

содержит только неповторяющиеся значения из обоих исходных рядов

intersect(x, y)

возвращает вектор только с теми значениями, которые встречаются

и в векторе x, и в векторе y

setdiff(x, y)

возвращает вектор только с теми значениями вектора x, которые

не встречаются в векторе y

setequal(x, y)

проверяет, содержат ли векторы x и y идентичные элементы (не обязательно

в одинаковых позициях), и возвращает соответствующее логическое значение

Mod(x); abs(x)

возвращают модуль x

Многие математические функции принимают аргумент `na.rm = TRUE`, который позволяет игнорировать отсутствующие значения при выполнении вычислений.

Матрицы

t(x)

выполняет транспонирование матрицы x

diag(x)

возвращает диагональ матрицы x

%*%

оператор умножения матриц

solve(a, b)

находит x в уравнении $a \%*\% x = b$, где a и b – матрицы

solve(a)

выполняет инверсию матрицы a

rowsum(x)

рассчитывает суммы по каждой строке матрицы x или другого

схожего по структуре объекта; более быстрой версией этой

функции является `rowSums(x)`

colsum(x), colSums(x)

функции, аналогичные предыдущей, но работают со столбцами матриц

rowMeans(x)

рассчитывает средние значения по каждой строке матрицы x

colMeans(x)

функция, аналогичная предыдущей, но работает со столбцами матриц

Даты и время

[as.Date\(s\)](#), [as.POSIXct\(s\)](#)

конвертируют вектор *s* в объект класса *Date* или *POSIXct* соответственно;
объекты этих двух классов хранят даты (*Date* – только даты, без времени;
POSIXct – и даты, и время, включая временные зоны); подробнее см.
?DateTimeClasses

[format\(dt\)](#)

конвертирует дату *dt* в текст; по умолчанию такой текст будет представлен
в виде "2001-02-21"; возможны и другие форматы

Ниже перечислены основные форматы дат и времени, принятые в системе R

%a, %A

сокращенное и полное названия дней недели (на англ. языке)

%b, %B

сокращенное и полное названия месяцев (на англ. языке)

%d

день месяца (01-31)

%H

часы (00-23)

%[I](#)

часы (01-12)

%j

день года (001-366)

%m

месяц (01-12)

%M

минута (00-59)

%p

AM/PM-представление времени суток

%S

секунда (00-61)

%U

неделя (00-53); первое воскресенье года считается 1-м днем 1-ой недели

%w

день недели (0-6, воскресенье имеет позицию 0)

%W

неделя (00-53); первый понедельник года считается 1-м днем 1-ой недели

%y

год, без указания века (00-99); лучше не использовать

%Y

год, с указанием века

Построение графиков

[plot\(x\)](#)

график значений вектора *x*, упорядоченных вдоль оси *x*

[plot\(x, y\)](#)

график зависимости *y* от *x*

[hist\(x, breaks = ...\)](#)

```
# гистограмма частот значений переменной x; аргумент breaks =  
# можно использовать, чтобы изменить принятое по умолчанию  
# количество столбцов
```

barplot(x)

```
# столбчатая диаграмма значений вектора x; аргумент horiz=TRUE  
# позволяет изобразить столбики горизонтально
```

dotchart(x)

```
# если x – таблица данных, то выполнение этой команды приведет  
# к созданию т.н. диаграммы Кливленда
```

pie(x)

```
# круговая диаграмма
```

boxplot(x)

```
# график типа "коробочки с усами"
```

sunflowerplot(x, y)

```
# то же, что и plot(), однако точки с одинаковыми координатами изображаются  
# в виде "ромашек", количество лепестков у которых пропорционально количеству  
# таких точек
```

stripplot(x)

```
# изображает значения x вдоль единственной горизонтальной оси
```

coplot(x~y | z)

```
# график зависимости y от x для каждого интервала значений z (или для  
# каждого уровня фактора, если z – фактор)
```

interaction.plot(f1, f2, y)

```
# если f1 и f2 – факторы, эта функция создаст график со средними значениями y  
# в соответствии со значениями f1 (по оси x) и f2 (по оси y, разные кривые);  
# аргумент fun = ... позволяет выбрать и другие параметры  
# описательной статистики, помимо средней
```

matplot(x, y)

```
# график зависимости первого столбца значений y от первого столбца значений x,  
# второго столбца y от второго столбца x, и т.д.
```

fourfoldplot(x)

```
# изображает (в виде частей окружности) связь между двумя бинарными  
# переменными в разных совокупностях (объект x должен быть массивом с  
# размерностью dim=c(2, 2, k) или матрицей размером dim=c(2, 2) при k = 1)
```

assocplot(x)

```
# график Кохена-Френдли, который изображает то, насколько независимы значения  
# в столбцах и строках таблицы сопряженности размером 2x2
```

mosaicplot(x)

```
# мозаичный график остатков лог-линейной регрессии
```

pairs(x)

```
# если x – матрица или таблица данных, эта функция изобразит диаграммы  
# рассеяния для всех возможных пар переменных из x
```

plot.ts(x)

```
# изображает временной ряд; объект x должен принадлежать к классу ts  
# (time series)
```

qqnorm(x)

```
# изображает квантили значений x против квантилей, которые можно было бы  
# ожидать при условии, что x является нормально распределенной переменной;  
# эта функция очень удобна для проверки соответствия данных закону  
# нормального распределения (при нормальном распределении точки  
# на таком графике должны примерно укладываться вдоль прямой линии,
```

```

# проходящей из точки начала координат под углом 45 градусов)

qqplot(x, y)
# график зависимости квантилей y от квантилей x

contour(x, y, z)
# выполняет интерполяцию данных и создает контурный график; x и y должны быть
# векторами, а z - матрицей, причем такой, что
# dim(z) = c(length(x), length(y))

filled.contour(x, y, z)
# то же, что contour(), но заполняет области между контурами определенными
# цветами; легенда с расшифровкой цветов изображается автоматически

image(x, y, z)
# изображает исходные данные в виде квадратов, цвет которых определяется
# значениями x и y

persp(x, y, z)
# то же, что и image(), но в виде трехмерного графика

stars(x)
# если x - матрица или таблица данных, изображает график в виде "звезд" так,
# что каждая строка представлена "звездой", а столбцы задают длину
# сегментов этих "звезд"

symbols(x, y, ...)
# изображает различные символы в соответствии с координатами, заданными x и y
# (квадраты, круги, треугольники, и т.п.); размеры и цвета этих символов
# задаются соответствующими аргументами функции

termplot(mod.obj)
# изображает частные эффекты переменных из регрессионной модели mod.obj;

# Многие графические функции управляют при помощи следующих аргументов:

add = FALSE
# если TRUE, то новый график будет прорисован поверх
# предыдущего (если он существует)

axes = TRUE
# если FALSE, оси графика и рамка вокруг него не будут нарисованы

type="p"
# задает тип графика: "p" - точки, "l" - линии, "b" - точки,
# соединенные линиями; "o" - то же, но точки представлены открытыми кружками,
# "h" - вертикальные линии; "s" - ступенчатые линии

xlim = , ylim =
# устанавливают верхний и нижний лимиты значений координатных осей, например:
# xlim=c(1, 10) или xlim=range(x)

xlab = , ylab =
# задают названия соответствующих осей (в виде текстовых векторов)

main =
# главный заголовок графика (должен быть текстовым вектором)

sub =
# второстепенный заголовок график (изображается более мелким шрифтом)

```

Графические функции низкого уровня

```

points(x, y)
# добавляет к графику точки с координатами x и y; можно использовать аргумент
# type = ... (см. выше), чтобы установить определенный вид этих точек

```

```

lines(x, y)
# то же, но для линий

text(x, y, labels, ...)
# добавляет к графику текст, указанный при помощи аргумента labels, в место
# с координатами x и y

mtext(text, side = 3, line = 0, ...)
# добавляет к графику текст, заданный аргументом text; сторона графика,
# куда будет добавлен текст, определяется параметром side (см. axis() ниже);
# аргумент line определяет, как близко к самому графику должен находиться
# добавляемый текст

segments(x0, y0, x1, y1)
# изображает линию, проходящую из точки с координатами (x0, y0) в точку
# с координатами (x1, y1)

arrows(x0, y0, x1, y1, angle= 30, code=2)
# то же, что и segments(), но для стрелок; головка стрелки будет изображена
# в точке с координатами (x0, y0) если code = 1, в точке (x1, y1) если
# code = 2, и с обеих сторон стрелки если code = 3; аргумент angle
# управляет тем, насколько остра головка стрелки

abline(a, b)
# рисует линию, заданную уравнением  $y = a + b \cdot x$ 

abline(h = y)
# рисует горизонтальную линию, отходящую от координаты y

abline(v = x)
# рисует вертикальную линию, отходящую от координаты x

abline(lm.obj)
# изображает регрессионную прямую, параметры которой хранятся в объекте
# lm.mod; объект lm.obj создается предварительно при помощи функции lm()

rect(x1, y1, x2, y2)
# рисует прямоугольник, чьи левая, правая, нижняя и верхняя грани задаются
# координатами x1, y1, x2 и y2 соответственно

polygon(x, y)
# рисует полигон, чьи координаты хранятся в объектах x и y

legend(x, y, legend)
# добавляет легенду в точке (x, y) с символами, которые задаются при помощи
# аргумента legend

title()
# добавляет к графику заголовок (также можно добавить подзаголовок)

axis(side, vect)
# добавляет к графику ось внизу (side = 1), слева (2), сверху (3),
# или справа (4)

rug(a)
# изображает данные из объекта a на оси X в виде небольших вертикальных линий

locator(n, type = "n", ...)
# возвращает координаты (x, y) того места на графике, куда пользователь
# кликнул мышкой n раз; также может нарисовать определенные символы
# (type = "p") или линии (type = "l") в этом месте в соответствии
# с графическими параметрами, предварительно заданными при помощи
# функции parameters (...); по умолчанию ничего не рисуется (type="n")

```

Управление графическим параметрами

```

# Многие из перечисленных ниже параметров могут быть установлены глобально
# при помощи команды par(...) или локально в виде аргументов
# графических функций

adj
# контролирует выравнивание текста (0 - по левому краю, 0.5 - по центру,
# 1 - по правому краю)

bg
# устанавливает цвет фона (например, bg = "red"); список 657 доступных цветов
# можно посмотреть при помощи команды
colors\(\)

bty
# управляет типом рамки вокруг графика (доступные опции: "o", "l", "7", "c",
# "u", "j" - рамка будет напоминать соответствующие символы); при bty = "n"
# рамка нарисована не будет

sex
# управляет размером шрифта и символов; следующие параметры имеют аналогичное
# действие в отношении: чисел на координатных осях - sex.axis,
# меток осей - sex.lab, основного заголовка графика - sex.main,
# и подзаголовка - sex.sub

col
# задает цвет символов и линий (в виде названий цветов "red", "blue",
# и т.п. - см. colors(), или в виде "#RRGGBB" - см. rgb(), hsv(),
# gray(), и rainbow()); так же, как и для sex, имеются варианты
# col.axis, col.lab, col.main, col.sub

font
# целое число, задающее стиль текста (1: нормальный, 2: курсив, 3: жирный,
# 4: жирный курсив); как и для sex, имеются варианты font.axis, font.lab,
# font.main, font.sub

las
# целое число, задающее ориентацию меток осей (0: параллельно осям,
# 1: горизонтально, 2: перпендикулярно, 3: вертикально)

lty
# управляет типом линий; может выражаться целым числом или просто
# соответствующим названием (1 или "solid" - сплошная; 2 или
# "dashed" - прерывистая; 3 или "dotted" - в виде точек, и т.д., до 6)

lwd
# число, задающее ширину линии (по умолчанию 1)

mar
# вектор из четырех чисел, определяющих пространство между осями
# и границей графика, в виде c(bottom, left, top, right);
# по умолчанию mar = c(5.1, 4.1, 4.1, 2.1)

mfcol
# вектор в виде c(nr, nc), позволяющий разбить окно графического
# устройства на nr строк и nc столбцов; каждый новый график будет
# потом последовательно нарисован в соответствующих ячейках
# получившихся столбцов

mfrow
# то же, но графики будут нарисованы прострочно

pch
# управляет типом символа; задается либо целым числом от 1 and 25, либо любым
# символом, заключенным в кавычки ""

```

```

ps
# целое число, определяющее размер (в пикселах) текста и символов

pty
# текстовое выражение, определяющее форму пространства, в котором будет
# нарисован график, например, "s" - квадрат

tck
# число, определяющее длину насечек на координатных осях; при tck = 1 будет
# нарисована координатная сетка

xaxt
# если xaxt="n", ось X не будет нарисована (полезно использовать в связке с
# axis(side = 1, ...))

yaxt
# то же для оси Y (полезно использовать в связке с axis(side = 1, ...))

```

Lattice-графика

```

xyplot(y ~ x)
# график зависимости y от x (имеется большой набор опций)

```

```

barchart(y ~ x)
# столбчатая диаграмма

```

```

dotplot(y ~ x)
# диаграмма Кливленда

```

```

densityplot(~ x)
# график плотности распределения значений x

```

```

histogram(~ x)
# гистограмма значений x

```

```

bwplot(y ~ x)
# график типа "коробочки с усами"

```

```

qqmath(~ x)
# аналог функции qqnorm() (см. выше)

```

```

stripplot(y ~ x)
# аналог функции stripplot(x) (см. выше)

```

```

qq(y ~ x)
# изображает квантили распределений x и y для визуального
# сравнения этих распределений; переменная x должна
# быть числовой, переменная y - числовой, текстовой, или
# фактором с двумя уровнями

```

```

splom(~ x)
# матрица диаграмм рассеяния (аналог функции pairs(), см.
# выше)

```

```

levelplot(z ~ x*y | g1*g2)
# цветной график значений z, координаты которых заданы
# переменными x и y (очевидно, что x, y и z должны иметь
# одинаковую длину); g1, g2... (если присутствуют) -
# факторы или числовые переменные, чьи значения автоматически
# разбиваются на равномерные отрезки

```

```

wireframe(z ~ x*y | g1*g2)
# функция для построения трехмерных диаграмм рассеяния
# и плоскостей; z, x и y - числовые векторы; g1, g2...
# (если присутствуют) - факторы или числовые переменные,
# чьи значения автоматически разбиваются на равномерные отрезки

```

```
cloud(z ~ x*y | g1*g2)
# трехмерная диаграмма рассеяния
```

Статистические модели

```
nlm(f, p)
# минимизирует функцию f, используя алгоритм типа ньютоновского;
# p - вектор с исходными значениями параметров функции
```

```
lm(formula)
# рассчитывает линейную регрессионную модель; формула модели formula обычно
# представлена в виде y ~ x1 + x2 + ...,
# где y - переменная-отклик, x1, x2, ... - предикторы
```

```
glm(formula, family = ...)
# рассчитывает логистическую регрессионную модель; аргумент
# family служит для указания функции распределения остатков модели
```

```
nls(formula)
# рассчитывает параметры нелинейной регрессионной модели по методу
# наименьших квадратов
```

```
spline(x, y)
# выполняет интерполяцию точек с координатами (x, y) кубическим сплайном
```

```
loess(formula)
# используя локальное сглаживание, выполняет подгонку
# полиномиальной плоскости к массиву точек, заданной одним
# или несколькими предикторами
```

```
# Многие функции, рассчитывающие статистические модели, имеют
# общие аргументы, в частности:
```

```
data = ...
# таблица данных, в которой хранятся переменные, указанные в
# формуле функции
```

```
subset = ...
# указывает, какие именно значения переменных должны принимать
# участие при расчете модели
```

```
na.action =
# указывает, как следует поступать с отсутствующими значениями;
# например, для игнорирования таких значений применяется
# na.action = "na.omit";
```

```
# Для рассчитанных моделей часто применимы следующие функции
# общего назначения:
```

```
predict(fit, ...)
# возвращает либо те значения, которые модель fit предсказывает
# на основе исходных данных, либо те значения, которые модель
# предсказывает для новых, заданных пользователем данных
```

```
df.residual(fit)
# возвращает число степеней свободы для остатков модели
```

```
coef(fit)
# возвращает рассчитанные коэффициенты модели (в ряде случаев,
# вместе с их стандартными ошибками)
```

```
residuals(fit)
# возвращает остатки модели
```

```
deviance(fit)
# возвращает девиату модели
```

```
fitted(fit)
# возвращает те значения, которые модель fit предсказывает
# на основе исходных данных

logLik(fit)
# рассчитывает логарифм правдоподобия и число параметров модели

AIC(fit)
# возвращает информационный критерий Акайке для модели fit
```

Избранные статистические тесты

```
aov(formula)
# выполняет дисперсионный анализ

anova(fit,...)
# выполняет дисперсионный анализ (или анализ девиат) для одной
# или нескольких подогнанных моделей fit

density(x)
# рассчитывает ядерные плотности вероятностей для значений x

binom.test()
# выполняет точный тест простой гипотезы о вероятности успеха
# в испытаниях Бернулли (другими словами, позволяет проверить,
# согласуются ли эмпирические частоты появления какого-то
# исхода испытаний с ожидаемыми)

pairwise.t.test()
# выполняет попарные сравнения несколько независимых или
# зависимых выборок (во втором случае необходим аргумент
# paired = TRUE), при одновременной поправке P-значений на
# число сравнений (метод поправки определяется аргументом
# p.adjust.method)

prop.test()
# позволяет проверить гипотезу о том, что частоты какого-либо
# признака равны во всех анализируемых группах (или о том,
# что они равны определенным значениям)

t.test()
# выполняет тест Стьюдента для двух независимых или зависимых
# выборок (во втором случае необходим аргумент paired = TRUE);
# возможно также сравнение выборочной средней с априори выбранной
# величиной (эта величина указывается при помощи аргумента mu)

# Для того, чтобы узнать о других статистических тестах,
# реализованных в R, выполните команду
help.search("test")
```

Программирование

```
function(список аргументов) {выражение}
# позволяет создать пользовательскую функцию

if(условие) {выражение}
# проверяет выполнение условия; если оно выполняется, выполняет
# выражение, указанное в фигурных скобках

if(условие) {выражение 1} else {выражение 2}
# проверяет выполнение условия; если оно выполняется, выполняет
# выражение 1, иначе - выражение 2

for(i in 1:n) {выражение}
```

```
# повторяет выражение, указанное в фигурных скобках, определенное
# число раз (в этом примере n раз)

while(условие) {выражение}
# исполняет выражение, указанное в фигурных скобках, до тех пор, пока
# условие выполняется

do.call(имя функции, аргументы)
# исполняет функцию, обращаясь к ней по имени; можно указать
# конкретные аргументы, которые будут управлять поведением функции
```

--

[Создано при помощи Pretty R на caïme inside-R.org](http://caïme.inside-R.org)